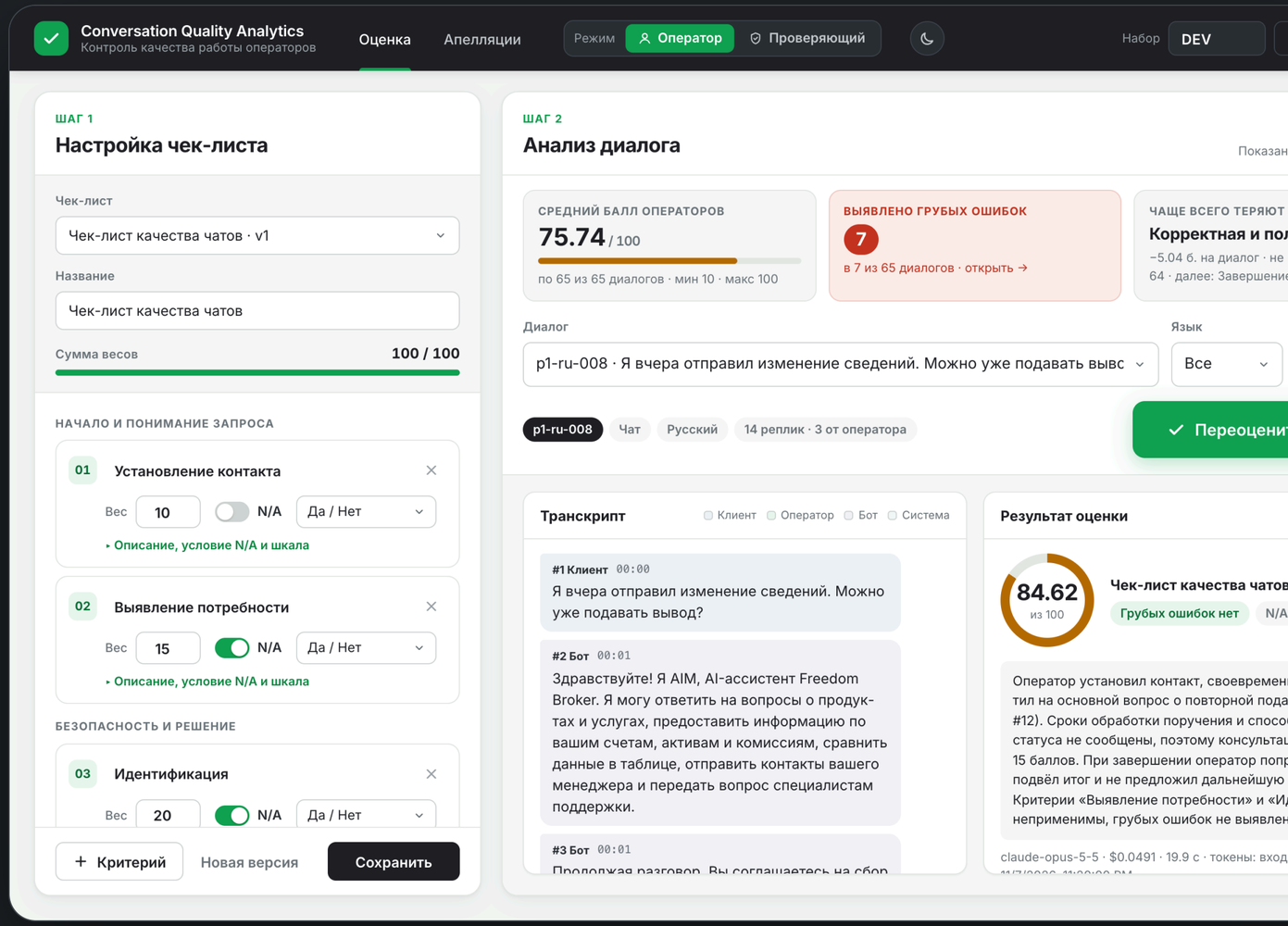
Conversation Quality Analytics

Автоматическая оценка работы операторов по чек-листу: чаты и звонки на русском, казахском и английском — с обоснованиями, апелляцией и пересмотром.

65 / 65	10 / 10	$0.054	≈ 20 с
DEV оценено, status ok	PROD FBroker, status ok	средняя стоимость чата	на оценку одного чата



КОМАНДА

room-801

Freedom Broker · CQA Hackathon

ЗАДАЧА → РЕШЕНИЕ

Оба контура из гайда работают в одном интерфейсе

Контур 1 · Оценка

- **Чек-лист настраивается** — критерии, веса, тип ответа, N/A, шкалы; новая версия применяется сразу
- **Оценка выбранного диалога** — RU / KZ / EN, бот и системные события как контекст
- **Результат** — ответ, баллы, вес, обоснование, реплика-доказательство, проверенный участок, причина N/A
- **Итог по правилам** — перераспределение веса N/A, округление, грубая ошибка отдельно

Контур 2 · Работа с результатом

- **Карточка диалога** — ссылка из обоснования ведёт к реплике; у звонков — плеер и перемотка
- **Апелляция** — оператор оспаривает критерий с комментарием
- **Решение проверяющего** — подтвердить или изменить с причиной, итог пересчитывается сразу
- **Плюс для бизнеса** — дашборд руководителя: средний балл, грубые ошибки, слабое место команды

АРХИТЕКТУРА

Чистый фронтенд + локальный сервер

БРАУЗЕР

HTML · CSS · JS

- без сборки и фреймворков
- всё в корне `#cqa-app`, адрес API — в `CQA_CONFIG`
- готов к HTML web resource **Dynamics 365**

`/api/*`

NODE.JS · BACKEND/

Локальный сервер

- обход CORS: запросы к API организаторов идут с сервера
- mp3 звонка из страницы плеера `audio_url`
- ключ модели не попадает в браузер
- чек-листы, оценки, апелляции — JSON

GET ?GROUP_ID=

API организаторов

- DEV · FBroker · Swap
- проверка isSuccess и total_records

ANTHROPIC API

Claude Opus 5.5

- строгая JSON-схема ответа
- проверка по чек-листу + 1 повтор

СДАЧА РЕЗУЛЬТАТОВ

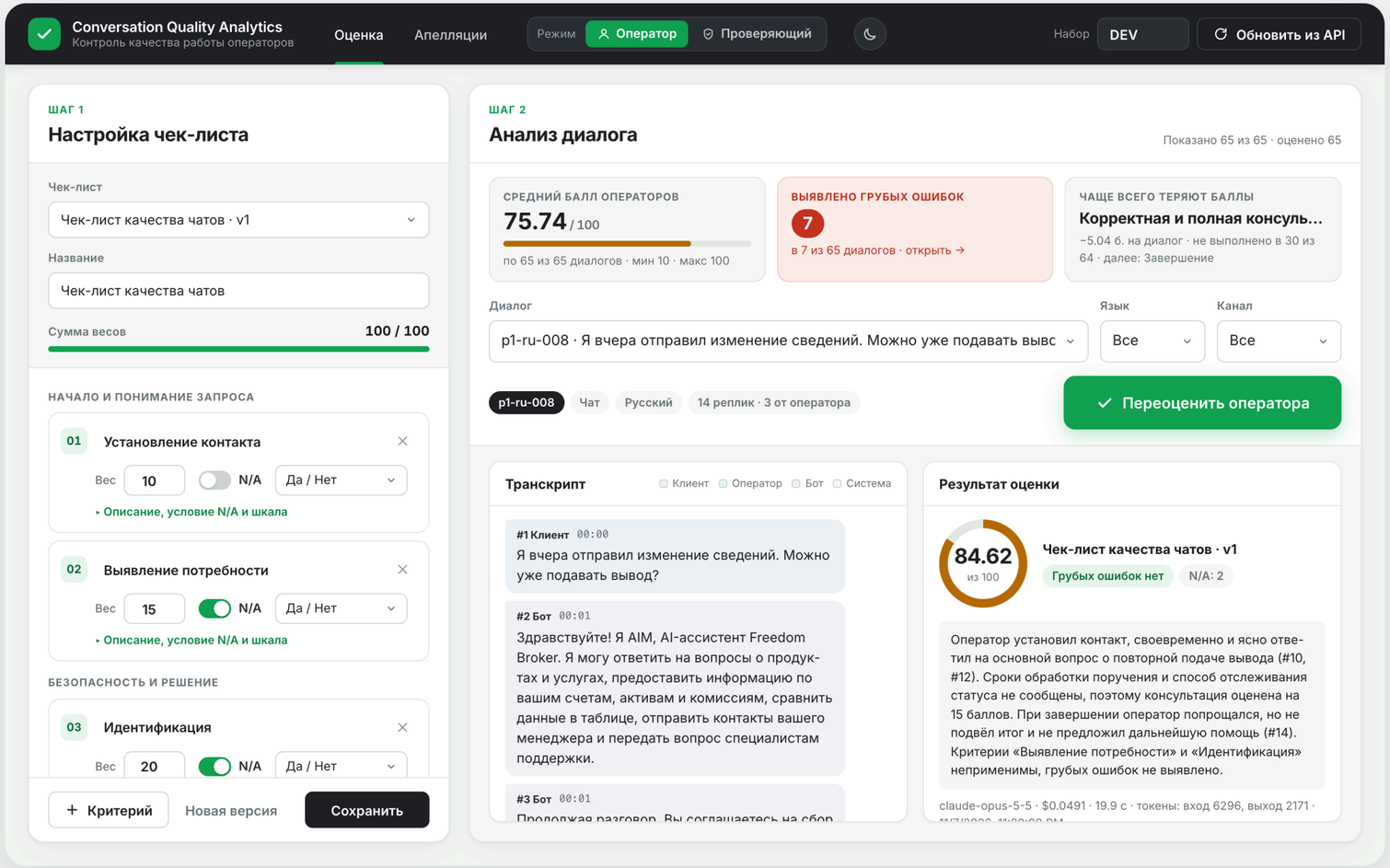
generate_results.js → `results/dev/results.json` · `results/prod/results.json`

- JSON 1.0 из приложения · пауза и повторы при 429 · прогресс после каждого диалога · проверка формата перед записью

ДЕМО · 1 МИНУТА

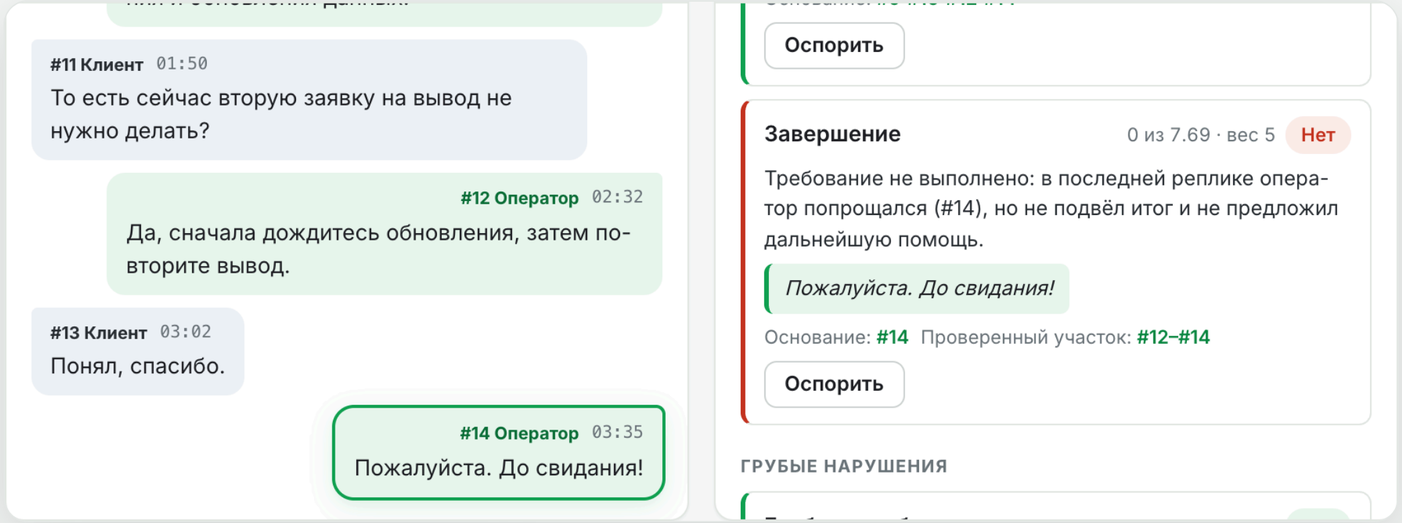
Интерфейс и выбранный диалог

1. Чек-лист слева
2. Дашборд руководителя
3. Выбор диалога
4. Транскрипт
5. Итог и обоснования



ДЕМО · 2 МИНУТЫ

Каждый вывод — со ссылкой на реплику





Итог 84.62 при двух N/A: веса «Выявления потребности» и «Идентификации» распределены между применимыми критериями.

- **Клик по обоснованию** подсвечивает реплику в чате и прокручивает к ней
- **Пропущенное действие** — указан проверенный участок #12–#14
- **N/A объясняется**: «данных для ответа хватало с самого начала»
- **Номера реплик** — с 1 по всему транскрипту, как `evidence_turns` в JSON 1.0

ИТОГОВЫЙ БАЛЛ

Итог считает код, а не модель

1. **Да** — полный вес, **Нет** — 0, шкала — пропорционально
   верхняя ступень даёт полный вес критерия
2. **N/A** исключается, его вес делится между применимыми
   максимум всегда остаётся 100
3. Округление до двух знаков
   нет применимых критериев → итог не рассчитывается
4. **Грубая ошибка** — отдельный флаг
   `critical_flag` не обнуляет итог

Пример из приложения организаторов

`contact ✓ · discovery N/A · identification N/A`
`consultation 15 · empathy ✓ · dialogue 7 · closure ✓`

применимый вес 65 → баллы 10 + 15 + 15 + 10.5 + 5 = 55.5

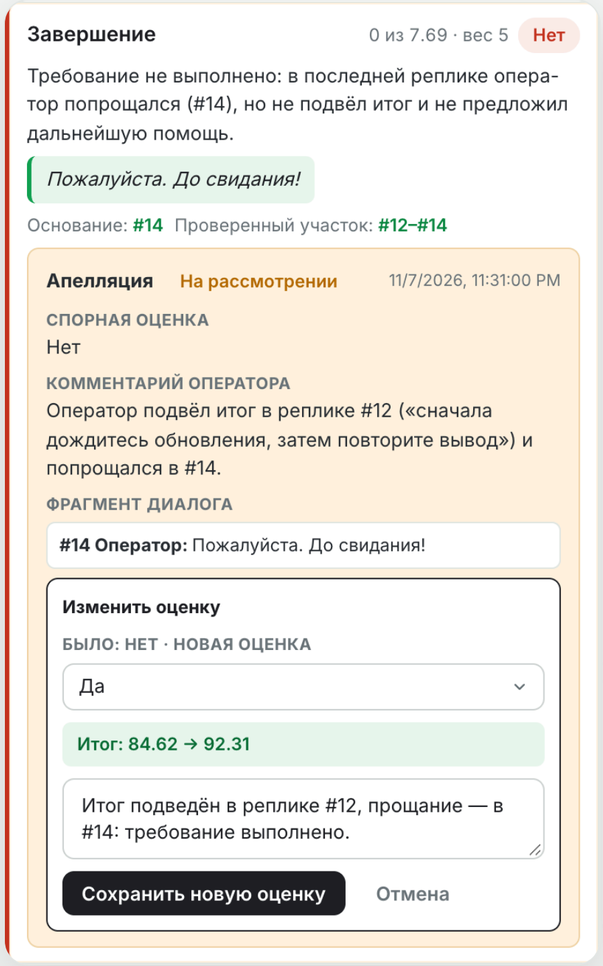
85.38 совпадает с эталоном

Проверено на реальных данных

Тест стабильности: 9 диалогов RU / KZ / EN, **19 проверок** расчёта на каждом — включая стресс «всё N/A». Бэкенд и фронтенд дают одинаковый итог.

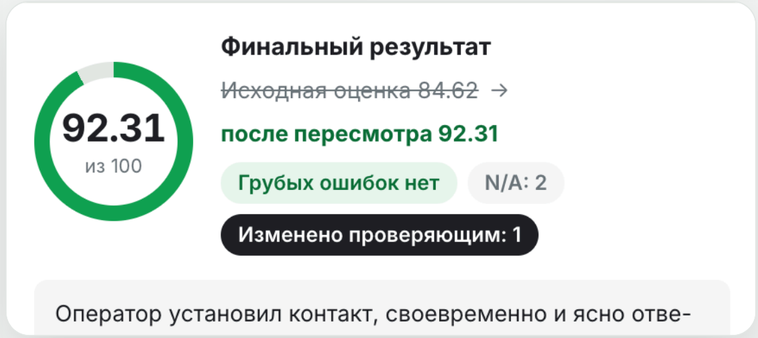
ДЕМО · 1 МИНУТА

Апелляция и решение проверяющего



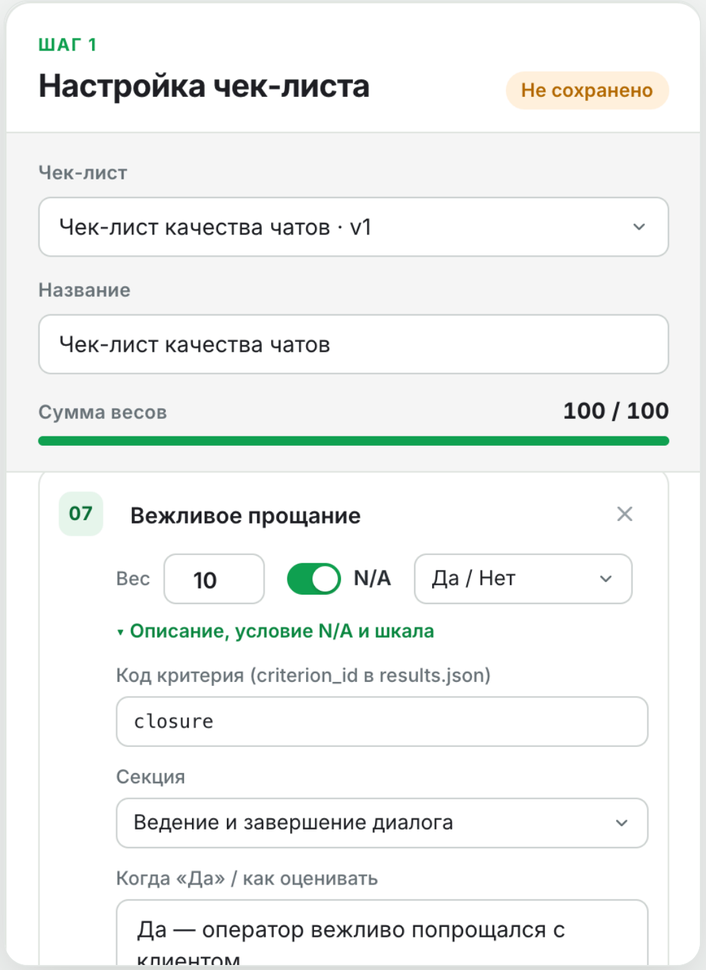
- **Оператор** оспаривает «Завершение»: итог подведён в #12
- **Проверяющий** видит спорную оценку, комментарий и фрагмент диалога
- Выбирает новую оценку, указывает причину — **итог пересчитан сразу**
- В карточке остаются исходная оценка, решение и финальный результат





ДЕМО · 1 МИНУТА

Изменение критерия — сразу в оценке



- **Промпт модели собирается из чек-листа**: названия, описания, веса, N/A и шкалы не захардкожены
- «Оценить» сохраняет несохранённые правки и оценивает по новой версии
- **Код критерия** редактируется — для замены критерия по кодам организаторов
- «Новая версия» хранит v1 и v2 — можно показать «было / стало»

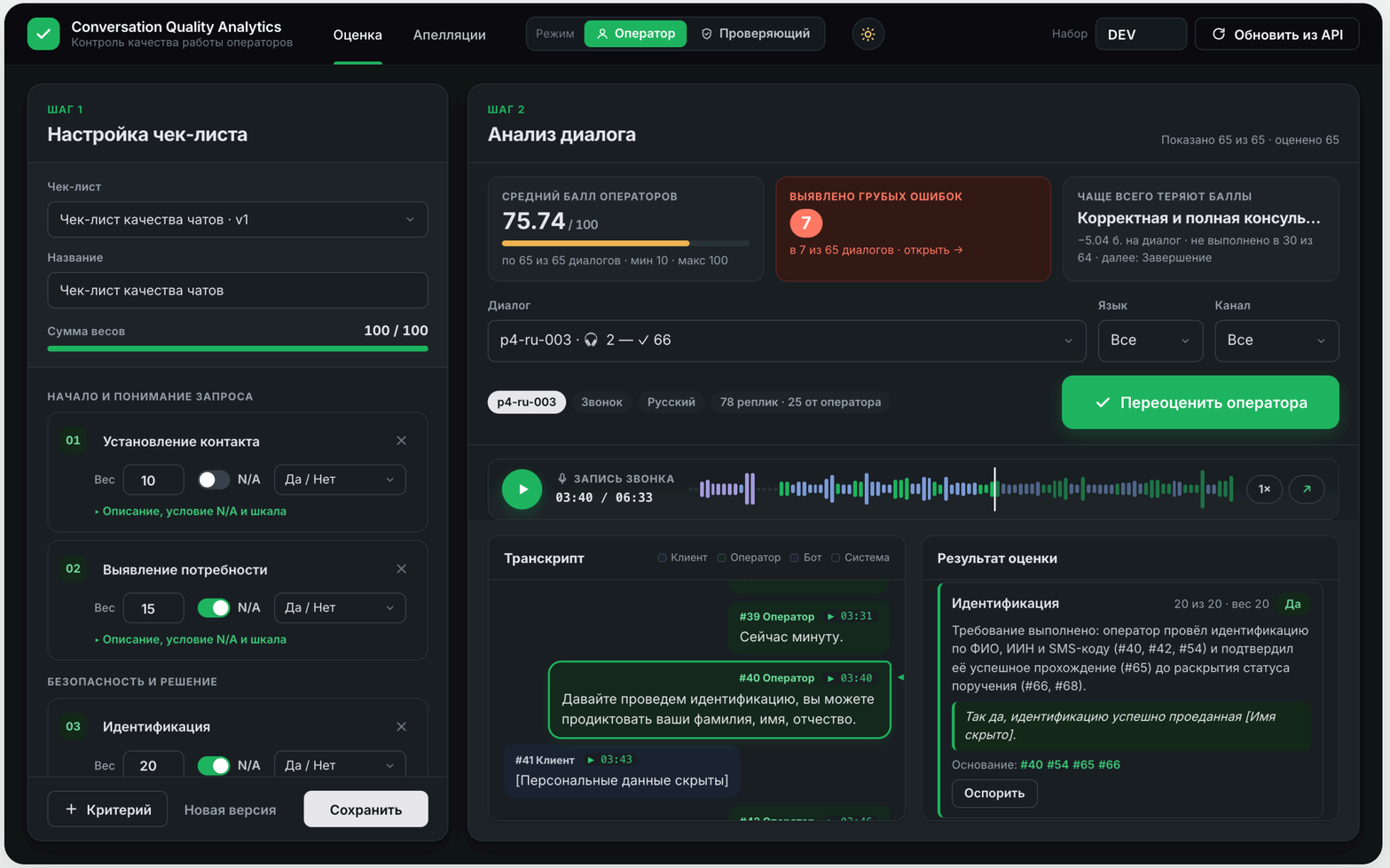
Проверено вживую на p1-ru-001

Правило «Завершения» → «вежливо попрощался», веса 06/07 → 10/10.

было: Нет · 93.75 **стало: Да · 100** **модель вернула вес 10**

ЗВОНКИ · VOICE CALL

Плеер записи и перемотка к доказательству



- **Шкала — активность речи** по транскрипту: высота — сколько сказано, цвет — оператор / клиент
- **Клик по обоснованию** подсвечивает реплику и перематывает запись
- Звучащая реплика отмечается в чате
- Расшифровка: обрывы и слова-паразиты **не снижают** оценку
- Тёмная тема — по системе или кнопкой

МОДЕЛЬ

Как мы удерживаем LLM в рамках

- **«Нет» ≠ N/A**: «Нет» — действие было нужно и пропущено, N/A — выполнено условие неприменимости
- **Анти-галлюцинация**: засчитывается только то, что явно подтверждено репликой
- **Строгая JSON-схема** (structured outputs) — модель не может вернуть другую структуру
- **Проверка по чек-листу**: типы value, ступени шкалы, номера реплик → при нарушении 1 повтор с перечнем ошибок
- **Корпоративный стиль** обоснований, без эмоциональных оценок

```
// запись JSON 1.0 · p1-kz-001
{ "conversation_id": "p1-kz-001",
  "status": "ok",
  "criteria": [
    { "criterion_id": "contact",
      "value": true,
      "rationale": "Требование выполнено:
        поздоровался на казахском (#10).",
      "evidence_turns": [10] },
    { "criterion_id": "identification",
      "value": null, … },
    … ],
  "score": { "final_score": 93.75,
             "critical_flag": false } }
```

Сбой модели → `status: "error"`, пустой `criteria`, `score: null` и причина — как требует приложение.

РЕЗУЛЬТАТЫ

Все диалоги оценены, формат проверен

НАБОР	ДИАЛОГОВ	СТАТУС	СТОИМОСТЬ	ЗА ДИАЛОГ
DEV · 60 чатов + 5 звонков	65	65 ok	$3.51	$0.054
PROD · FBroker	10	10 ok	$0.52	$0.052
Всего	**75**		**$4.03**	**$0.054**

`results/dev/results.json` и `results/prod/results.json` — JSON 1.0: ID совпадают с API, каждая запись проверена валидатором, итоги совпадают с пересчётом.

МОДЕЛЬ

Claude Opus 5.5

`claude-opus-5-5 · external_api`

СТОИМОСТЬ ОДНОЙ ОЦЕНКИ

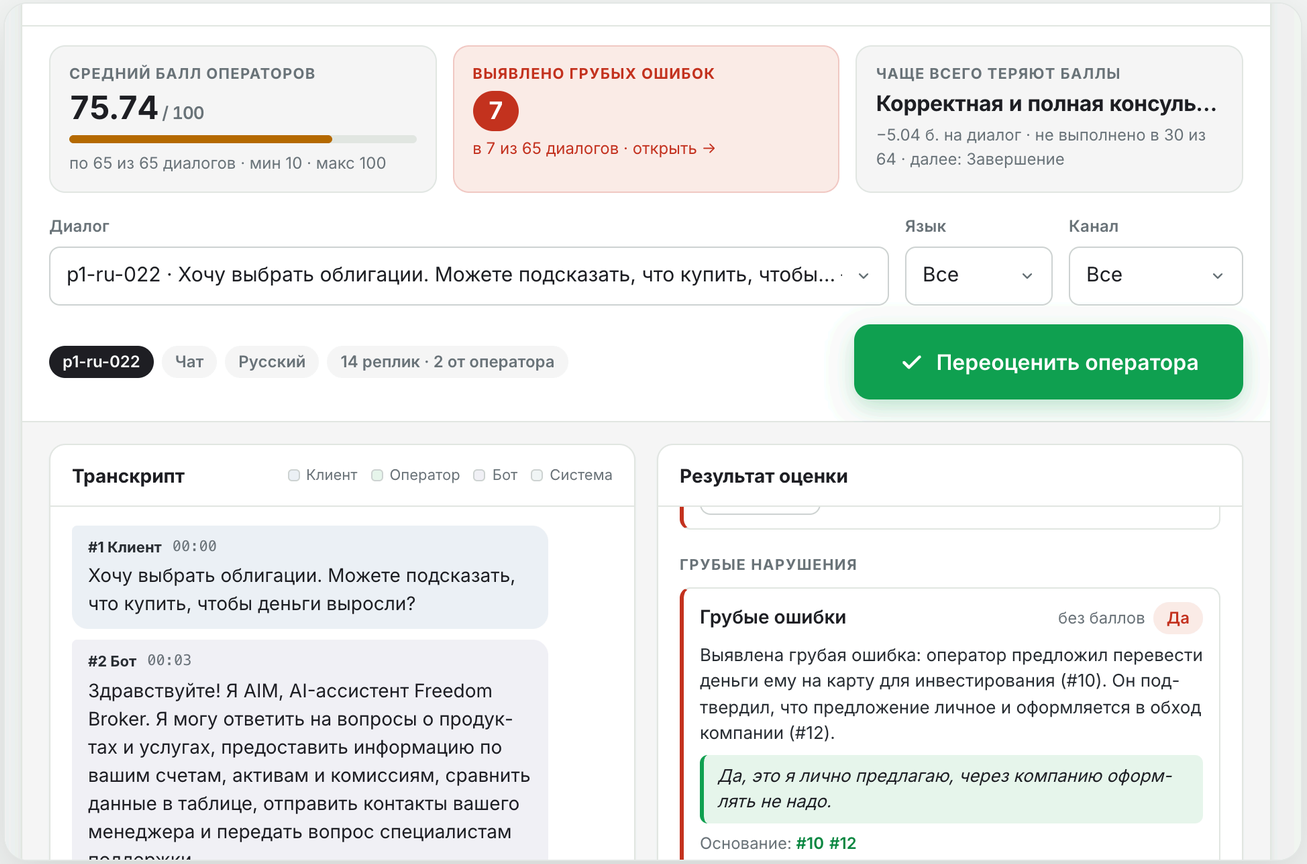
$0.054 · ≈ 20 с на чат

Один вызов модели; системный промпт с чек-листом кэшируется. Звонок ~80 реплик — $0.08–0.12.

8 грубых ошибок найдено (7 в DEV, 1 в PROD) — каждая подтверждается текстом реплики

ГОТОВНОСТЬ К ПРОДУКТУ

Для руководителя и для эксплуатации



- **Дашборд**: средний балл, грубые ошибки (клик — к диалогу), критерий, где теряют больше всего — по финальным оценкам с учётом апелляций
- **Не падает**: таймауты, обрыв сети, неполный JSON — понятное уведомление и «Повторить», без белого экрана
- **Скелетная загрузка** вместо спиннеров
- **Dynamics 365**: один корневой элемент, стили изолированы, адрес API настраивается

ИТОГ

Запуск за три команды

```
# 1. зависимости и настройки
cd backend && npm install
cp .env.example .env   # ключ API и ссылка организаторов

# 2. набор и сервер
npm run fetch && npm start   # → http://localhost:3000

# 3. результаты для сдачи
npm run results:dev
node generate_results.js --prod --group=FBroker
```

github.com/MedgatAuken/DataChallengeFreedom

README: архитектура, запуск, формат, стоимость, ограничения

Честно об ограничениях

- Звонки оцениваются по транскрипту; интонацию и дикцию без анализа звука не оцениваем
- LLM недетерминирована: в пограничных случаях оценка критерия может отличаться между прогонами — для этого есть апелляция
- Хранилище — JSON на одном сервере; роли — симуляция без авторизации